Guía práctica: Desarrollo de software con Claude Code

Bloques 1 y 2: Arranque del proyecto, base de datos y API REST

Ingeniería de Software · Universidad Iberoamericana León

14 de octubre de 2026 · M.I. Ricardo Medrano

1. Introducción

Al terminar los bloques 1 y 2 tu proyecto correrá en tu computadora con un solo comando: una base de datos PostgreSQL con migraciones, una API REST en FastAPI que aplica tus reglas de negocio y un front web servido por nginx, todo en contenedores Docker. Lo construirás con **Claude Code**, partiendo de tu plan de proyecto y de tu prototipo.

Objetivos de aprendizaje

1. Usar Claude Code como agente de desarrollo: darle contexto, revisar sus planes y verificar lo que hace.
2. Convertir el plan de proyecto en instrucciones permanentes para el agente (`CLAUDE.md`).
3. Levantar una arquitectura de tres capas con Docker Compose.
4. Diseñar el modelo de datos, versionarlo con migraciones y exponerlo como API REST.
5. Conectar herramientas externas a Claude mediante MCP, aplicando el principio de menor privilegio.
6. Usar Git como red de seguridad y la terminal para verificar el trabajo del agente.

Cómo usar esta guía

El profesor demuestra cada paso en vivo con un **proyecto espejo**: *LabPresta*, un sistema de préstamo de equipo de laboratorio con tres entidades (Equipo, Solicitante, Préstamo), tres reglas de negocio y tres pantallas. Tú repites el mismo paso **en tu propio proyecto**.

Cada paso tiene la misma estructura:

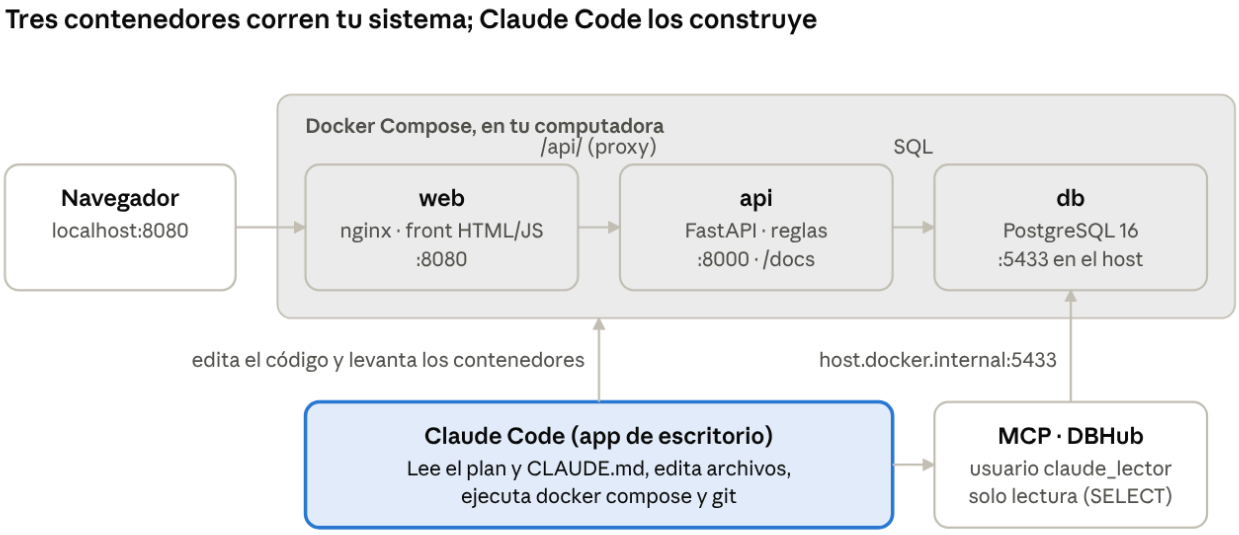
- **Prompt**: el texto que escribes en Claude Code. Cópialo y ajusta los nombres a tu proyecto donde se indique.
- **Qué observar**: lo que debes notar mientras Claude trabaja.
- **Verificación**: cómo compruebas tú, no Claude, que el resultado es correcto.

> Regla de oro: Claude propone y ejecuta; **tú decides y verificas**. Nunca aceptes un cambio que no entiendes.

Alcance mínimo obligatorio (MVP)

- 2 o 3 entidades de tu plan, cada una con su CRUD completo.
- Las pantallas de tu prototipo funcionando contra la API real.
- Si tu proyecto tiene un componente de IA, en este MVP se representa con un **endpoint simulado** que devuelve una respuesta fija con la misma forma que la real.

Arquitectura objetivo



Arquitectura objetivo: 3 contenedores + Claude Code + MCP

El navegador solo habla con nginx; nginx reenvía `/api/` a FastAPI, que es la única capa que escribe en PostgreSQL. Claude Code construye todo y consulta la base mediante MCP con un usuario que no puede modificarla.

Mapa de la práctica

Bloque	Tema	Duración en clase	Resultado
Tarea previa	Instalación y preparación del repositorio	En casa	Herramientas verificadas, repositorio con `docs/` y `prototipo/`
1	Fundamentos y arranque del proyecto	2 h	`CLAUDE.md` y stack de tres contenedores funcionando

Bloque	Tema	Duración en clase	Resultado
2	Base de datos, API REST y conector MCP	2 h	Modelo con migraciones, API con reglas de negocio, MCP de solo lectura
3	Front end desde el prototipo	2 h	Pantallas conectadas a la API, verificadas en el navegador
4	Mantenimiento y entrega	2 h	Cambios de requerimiento, corrección de errores, pruebas y documentación

Esta guía cubre la tarea previa y los bloques 1 y 2.

2. Tarea previa: antes de la primera clase

La instalación se hace en casa, no en clase: en la prueba piloto, solo Docker tomó una hora. Llega a la clase 1 con todo instalado, el repositorio preparado y la evidencia entregada.

2.1 Software requerido

Herramienta	Para qué sirve	Windows	Mac
App de escritorio de Claude	Ejecutar Claude Code (pestaña **Code**)	Instalador x64 o ARM64 desde claude.ai/download	Instalador .dmg desde claude.ai/download
Git	Control de versiones; red de seguridad ante cambios del agente	Git for Windows (git-scm.com)	`xcode-select --install` en Terminal, o git-scm.com
Docker Desktop	Correr la base de datos, la API y el servidor web en contenedores	Requiere WSL 2 y virtualización activada	Elige la versión para tu chip (Apple o Intel)

Necesitas una cuenta de Claude con un plan que incluya Claude Code (Pro o superior); el plan gratuito no lo incluye.

No necesitas instalar Python, PostgreSQL ni Node.js en tu computadora: todo corre dentro de contenedores. Si instalas Node.js por otro motivo en Windows, **no marques** la casilla *"Automatically install the necessary tools"*: instala varios GB de Visual Studio Build Tools que este proyecto no usa y puede quedarse detenido.

2.2 Pasos de instalación

1. **Solo Windows: activa WSL 2.** Abre PowerShell como administrador, ejecuta `wsl --install` y reinicia. Verifica la virtualización en *Administrador de tareas → Rendimiento → CPU → Virtualización: Habilitado*. Si dice deshabilitado, actívala en el BIOS de tu equipo.
2. **Instala Docker Desktop**, ábrelo y espera a que indique que el motor está corriendo (*Engine running*).
3. **Instala Git** y configura tu identidad (una sola vez por computadora):

   ```
   git config --global user.name "Tu Nombre"
   git config --global user.email "tu.correo@ejemplo.com"
   ```

4. **Instala la app de escritorio de Claude**, inicia sesión y abre la pestaña **Code**.

2.3 Prepara tu repositorio

Crea una carpeta para tu proyecto, de preferencia **sin espacios ni acentos** en el nombre (por ejemplo `mi-proyecto`), con esta estructura:

```
mi-proyecto/
├── docs/
│   └── plan_proyecto.pdf      (o .md)
└── prototipo/
    └── index.html
```

- **Plan de proyecto**: Claude Code lee Markdown y PDF directamente. Si tu plan está en Word, guárdalo como PDF (*Archivo → Guardar como → PDF*).
- **Prototipo**, según dónde lo hiciste:
  - Claude Design o artifact de Claude: descarga el HTML.
  - Google Stitch: exporta el código HTML.
  - Lovable, Vercel, Netlify u otro sitio publicado: descarga el código fuente, o guarda cada página desde el navegador.
  - Si solo tienes imágenes, coloca las capturas PNG de cada pantalla en `prototipo/`.

2.4 Verificación y evidencia

En una terminal ejecuta y toma captura del resultado:

```
git --version
docker --version
docker compose version
docker run hello-world
```

El último comando debe mostrar el mensaje *"Hello from Docker!"*. Después, en la pestaña **Code** de la app, selecciona tu carpeta y escribe: *"¿Qué archivos hay en esta carpeta?"*. Toma captura de la respuesta.

3. Conceptos clave de Claude Code

Claude Code no es un chat que te da código para copiar: es un **agente** que lee tus archivos, los edita y ejecuta comandos en tu computadora. Por eso lo importante no es escribir código, sino **darle buen contexto, revisar sus planes y verificar sus resultados**.

3.1 La sesión en la app de escritorio

En la pestaña **Code**, antes de enviar el primer mensaje, configura en el área del prompt:

Opción	Valor para esta práctica
Environment	**Local** (tu computadora)
Project folder	La carpeta de tu proyecto
Model	El predeterminado
Permission mode	Según el paso (ver 3.2)

Cada sesión tiene su propio historial. Puedes tener varias en la barra lateral.

3.2 Modos de permisos

Modo	Qué hace Claude	Cuándo usarlo
Manual	Pide permiso antes de editar cada archivo o ejecutar cada comando	Al inicio, para aprender qué hace
Plan	Explora y propone un plan; no modifica código	Antes de cualquier cambio que toque más de un archivo
Accept edits	Edita archivos sin preguntar; pregunta antes de otros comandos	Para ejecutar un plan que ya aprobaste
Auto	Trabaja sin pedir permisos rutinarios	No se usa en esta práctica
Bypass permissions	Sin permisos	Nunca en tu computadora

3.3 Contexto y sesiones

Todo lo que dices y lo que Claude lee se acumula en el **contexto** de la sesión. Un contexto largo y viejo produce respuestas peores y consume más de tu límite de uso.

- **Una sesión por tarea.** Al empezar una tarea nueva, abre una sesión nueva o escribe `/clear`.
- `/compact` resume la conversación cuando quieres seguir en la misma sesión.
- Pide tareas pequeñas y concretas, nunca *"hazme todo el sistema"*.

3.4 CLAUDE.md: la memoria del proyecto

`CLAUDE.md` es un archivo en la raíz del repositorio que Claude lee al iniciar **cada** sesión. Ahí viven el resumen del proyecto, el stack, las convenciones, las reglas de trabajo y los comandos frecuentes. Gracias a él, una sesión nueva "recuerda" el proyecto sin el historial anterior. Es el concepto más importante de la práctica: convierte tu plan de proyecto en instrucciones permanentes para el agente.

3.5 Herramientas de la sesión

Herramienta	Cómo se abre	Para qué
Comandos	Escribe `/` en el cuadro de texto	`/init`, `/clear`, `/compact`, `/mcp` y otros
Changes	Botón **Changes** o el indicador `+12 -1`	Ver el diff de lo que hizo Claude y **comentar línea por línea**; los comentarios se envían con `Ctrl + Enter` (`Cmd + Enter` en Mac)
Terminal	Botón **Terminal** o `Ctrl + ` `	Ejecutar comandos tú mismo en la carpeta del proyecto
Browser	`Ctrl + Shift + B` (`Cmd + Shift + B` en Mac)	Claude abre tu app en localhost, navega y toma capturas (bloque 3)

3.6 MCP: herramientas externas

MCP (Model Context Protocol) es el estándar con el que Claude se conecta a herramientas fuera de tus archivos. Hay dos tipos:

- **Conectores**: servicios en la nube ligados a tu cuenta (Google Calendar, GitHub, Notion). Se agregan con el botón **+** → **Connectors** o en **Settings → Connectors**.
- **Servidores MCP del proyecto**: herramientas que viajan con el repositorio, definidas en el archivo `.mcp.json`. Al abrir una sesión nueva, Claude Code te pide **aprobarlas**. En el bloque 2 crearás uno para consultar tu base de datos.

4. Bloque 1: Fundamentos y arranque del proyecto

Al terminar este bloque tendrás un `CLAUDE.md` con las reglas de tu proyecto y tres contenedores (base de datos, API y web) levantándose con un solo comando. Duración: 2 horas.

Paso 1.1: Crea la sesión

Abre la pestaña **Code**, elige **Local**, selecciona la carpeta de tu proyecto y pon el modo de permisos en **Manual**.

Qué observar: con el modo Manual, Claude te pedirá permiso antes de cada cambio. Así ves exactamente qué hace.

Paso 1.2: Red de seguridad con Git

Antes de que Claude toque cualquier archivo, abre la **Terminal** de la sesión (`Ctrl + ``) y ejecuta:

```
git init
git add .
git commit -m "Insumos: plan de proyecto y prototipo"
```

Qué observar: un commit es tu "guardar partida". Si algo sale mal, siempre podrás volver a este punto.

Paso 1.3: Comprensión antes que código

Prompt:

```
Lee docs/plan_proyecto (el archivo que esté en esa carpeta) y el prototipo
en la carpeta prototipo/. Explícame en no más de 15 líneas qué sistema vamos
a construir, qué entidades y reglas de negocio identificas y qué pantallas
tiene el prototipo. No crees ni modifiques ningún archivo.
```

Qué observar: Claude abre y lee los archivos por su cuenta; esa es la diferencia con un chat.

Verificación: compara su resumen con tu plan. Si confundió una entidad o una regla, corrígelo ahora en la conversación, antes de generar código.

Paso 1.4: Crea la memoria del proyecto (CLAUDE.md)

Escribe `/` en el cuadro de texto y elige **`/init`**. Si no aparece, escribe: *"Crea un archivo CLAUDE.md para este proyecto"*. Después complétalo:

Prompt:

```
Complementa CLAUDE.md con estas secciones, en español y en menos de 80 líneas:
1. Resumen del proyecto. El plan en docs/ es la fuente de verdad del alcance.
2. Stack fijo: FastAPI + SQLAlchemy 2 + Pydantic 2 + Alembic, PostgreSQL 16,
   front HTML/CSS/JS estático servido por nginx, Docker Compose con los
   servicios web, api y db.
3. Estructura de carpetas propuesta (api/, web/, docs/, prototipo/).
4. Convenciones: tablas y campos en español como en el plan, código en Python
   con type hints.
5. Reglas de trabajo: proponer un plan antes de cambios que toquen más de
   un archivo; no agregar dependencias sin preguntar; todo debe correr con
   docker compose up; no instalar nada en la máquina host.
6. Comandos frecuentes (levantar, detener, ver logs, migraciones).
```

Verificación: abre **Changes** y lee el archivo completo. Si algo no te convence, **comenta sobre la línea** en lugar de editarla a mano y envía con `Ctrl + Enter`. Cuando estés conforme, haz commit.

Paso 1.5: Modo plan para la estructura base

Cambia el modo de permisos a **Plan**.

Prompt:

```
Planea la estructura base del proyecto, sin lógica de negocio todavía:
- db: postgres:16 con volumen persistente, healthcheck y puerto 5433 en el host.
- api: Dockerfile con Python 3.12 y FastAPI, con un endpoint GET /health que
  verifique la conexión a la base de datos. En desarrollo, monta ./api/app como
  volumen y arranca uvicorn con --reload, con WATCHFILES_FORCE_POLLING=true
  para que detecte cambios en Windows.
- web: nginx sirviendo la carpeta web/ con una copia de prototipo/index.html,
  y un proxy de /api/ hacia el servicio api para evitar problemas de CORS.
- Variables de entorno en .env, con un .env.example versionado y .env en
  .gitignore. DATABASE_URL se construye en docker-compose.yml a partir de
  POSTGRES_USER, POSTGRES_PASSWORD y POSTGRES_DB, sin duplicar credenciales.
- Un .gitattributes que fuerce finales de línea LF en todo el repositorio.
```

Qué observar: Claude pedirá aprobación para las dependencias nuevas. No es casualidad: es la regla que pusiste en `CLAUDE.md`.

Verificación: revisa el plan con estas cinco preguntas antes de aprobarlo.

1. ¿Respeta el alcance y dice explícitamente qué **no** va a hacer?
2. ¿Pide aprobación para las dependencias nuevas?
3. ¿Incluye cómo verificar el resultado, con al menos un caso de falla?
4. ¿Cómo afecta mi trabajo diario? Por ejemplo, ¿tendré que reconstruir la imagen en cada cambio?
5. ¿Hay datos o secretos duplicados?

Si algo falta, pídelo en la conversación **sin salir del modo Plan**. Cuando estés conforme, apruébalo y cambia a **Accept edits** para que lo ejecute.

Paso 1.6: Levanta el stack

En la Terminal:

```
docker compose up --build
```

Verificación: abre en tu navegador:

Dirección	Qué debes ver
`http://localhost:8080`	Tu prototipo servido por nginx
`http://localhost:8000/docs`	La documentación Swagger de FastAPI
`http://localhost:8080/api/health`	`{"status":"ok","db":"up"}`, pasando por el proxy

Si algo falla, copia el error de la terminal y pégaselo a Claude tal cual.

Paso 1.7: Comprueba el modo desarrollo

Prompt:

```
Comprueba que el reload funciona: agrega "version": "0.1" a la respuesta de
/health y, sin reconstruir la imagen, verifica con curl que el cambio se refleja.
```

Verificación: la respuesta de `/health` cambia sin ejecutar `--build`. Desde ahora, cada cambio en el código de la API se aplica solo.

Paso 1.8: Commit hecho por Claude

Prompt:

```
Revisa los cambios con git status y git diff, y haz commit con un mensaje
descriptivo en español.
```

Verificación: `.env` **no** debe aparecer en el commit; solo `.env.example`.

Resultado esperado del bloque 1

```
mi-proyecto/
├── CLAUDE.md
├── docker-compose.yml
├── .env.example   .env (no versionado)   .gitignore   .gitattributes
├── api/
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/  (main.py con /health, db.py)
├── web/
│   ├── index.html  (copia del prototipo)
│   └── nginx.conf
├── docs/
└── prototipo/
```

5. Bloque 2: Base de datos, API REST y conector MCP

Al terminar este bloque tendrás tus entidades en PostgreSQL con migraciones versionadas, una API REST que aplica tus reglas de negocio y un servidor MCP con el que Claude consulta tu base sin poder modificarla. Duración: 2 horas.

Paso 2.0: Sesión nueva

Abre una **sesión nueva** en la misma carpeta, con el modo **Plan**. No continúes en la sesión del bloque 1.

Qué observar: sin el historial anterior, Claude conoce tu proyecto porque lee `CLAUDE.md`. Esa es la prueba de que la memoria del proyecto funciona.

Paso 2.1: Modelo de datos y migraciones

Prompt:

```
Planea el modelo de datos y las migraciones, sin endpoints todavía:
- Modelos SQLAlchemy 2 para las entidades del alcance de la Fase 1 del plan
  en docs/, con unicidad, llaves foráneas y los estados restringidos a sus
  valores válidos.
- Alembic configurado para leer DATABASE_URL, con una migración inicial
  autogenerada. Muéstrame la migración antes de aplicarla.
- Las migraciones se aplican automáticamente al arrancar el contenedor api.
- Un script de datos de ejemplo idempotente que use los mismos datos que ya
  tiene el prototipo en prototipo/.
```

Qué observar:

- Claude pedirá aprobar Alembic como dependencia nueva (regla de `CLAUDE.md`).
- Probablemente cree un script `entrypoint.sh` que corre las migraciones. El `.gitattributes` del bloque 1 lo protege del error de finales de línea en Windows.
- Tomar los datos de ejemplo del prototipo hace que en el bloque 3 el front muestre lo mismo que ya validaste.

Verificación: revisa la migración autogenerada antes de aplicarla.

1. ¿Están todas las tablas y columnas del plan, con el tipo correcto?
2. ¿Las columnas opcionales aceptan nulos y las obligatorias no? (En LabPresta, `fecha_devolucion` es nula mientras no se devuelve el equipo.)
3. ¿Hay restricciones de unicidad donde el plan las pide (clave de inventario, matrícula)?

4. ¿Las llaves foráneas apuntan a la tabla correcta?
5. ¿Los estados están limitados a sus valores válidos? Alembic a veces se equivoca con los tipos enum: revísalos con cuidado.

Paso 2.2: Comprueba la base con tus propios ojos

En la Terminal (usa el usuario y la base de tu `.env`):

```
docker compose exec db psql -U labpresta -d labpresta -c "\dt"
docker compose exec db psql -U labpresta -d labpresta -c "\d prestamo"
```

Fragmento	Significado
`docker compose exec db`	Ejecuta un comando dentro del contenedor `db`, que ya está corriendo
`psql`	Cliente de PostgreSQL; ya viene en la imagen, no lo instalas
`-U labpresta -d labpresta`	Usuario y base de datos (pueden llamarse igual, pero son cosas distintas)
`-c "\dt"`	Ejecuta un solo comando y sale. `\dt` lista las tablas; `\d <tabla>` describe una tabla

Verificación: con `\dt` debes ver tus tablas y `alembic_version`, donde Alembic guarda qué migración está aplicada. Con `\d <tabla>` compara columnas, nulos, índices y llaves foráneas contra tu plan.

> Si la salida no cabe en pantalla, psql la muestra con un paginador. **¿Ves `:` o `(END)`? Presiona `q`.** (Más en la sección 6.)

Paso 2.3: API REST con reglas de negocio

Abre una sesión nueva o escribe `/clear`, en modo **Plan**.

Prompt (ejemplo de LabPresta; sustituye las entidades y reglas por las de tu plan):

```
Planea los endpoints REST:
- CRUD de equipo (con filtros por estado y categoría) y de solicitante.
- POST /prestamos que aplique RN1 y cambie el equipo a prestado.
- GET /prestamos/activos que marque como atrasado lo que cumpla RN2.
- POST /prestamos/{id}/devolucion que aplique RN3.
Errores de negocio con códigos HTTP adecuados (409, 404, 422) y mensajes
en español. Esquemas Pydantic separados para entrada y salida.
```

Si tu proyecto tiene componente de IA, agrega al prompt:

```
- Un endpoint simulado para el modelo de IA que reciba la misma entrada que
  recibirá el modelo real y devuelva una respuesta fija con la misma
  estructura. Documéntalo como simulado.
```

Verificación en Swagger (`http://localhost:8000/docs`):

1. Crea, consulta, edita y elimina un registro de cada entidad.
2. Ejecuta el caso feliz de cada regla de negocio.
3. **Prueba negativa:** intenta violar una regla (en LabPresta, prestar un equipo ya prestado). Debe responder **409** con un mensaje en español.

Paso 2.4: Conector MCP de base de datos

Claude podrá consultar tus datos reales con un usuario de **solo lectura**: aprende a usarlo sin riesgo de modificar nada.

Prompt:

```
Configura un servidor MCP de PostgreSQL para este proyecto:
- Crea, mediante una migración de Alembic, un rol de solo lectura llamado
  claude_lector con permiso únicamente de SELECT.
- Agrega un .mcp.json en la raíz que ejecute la imagen Docker de DBHub
  (docker run -i --rm bytebase/dbhub) conectada a host.docker.internal:5433
  con ese usuario, para que el mismo archivo funcione en Windows y Mac.
  No uses npx.
- Anota en CLAUDE.md que el stack debe estar levantado para que el MCP funcione.
```

Qué observar: el servidor corre en Docker, así que respeta la regla de no instalar nada en tu computadora. Desde un contenedor, `localhost` es el propio contenedor; `host.docker.internal` es tu máquina, en Windows y en Mac.

Después:

1. **Abre una sesión nueva** para que Claude cargue el servidor.
2. **Aprueba** el servidor del proyecto cuando Claude Code lo pida.
3. Comprueba que está conectado: escribe `/mcp`, o pregunta *"¿Qué servidores MCP tienes disponibles y qué herramientas ofrece cada uno?"*.

Verificación con dos pruebas:

```
Usando el MCP de base de datos, dime cuántos registros hay por estado en la
tabla principal y cuáles cumplen la regla de atraso.
```

```
Usando el MCP, intenta cambiar el estado de un registro.
```

La primera debe responder con datos reales. La segunda **debe fallar** por falta de permisos: eso demuestra el principio de menor privilegio.

> La contraseña de `claude_lector` queda en `.mcp.json`. Es aceptable solo porque es una base local de desarrollo con un usuario de solo lectura; en un servidor real nunca se versionan credenciales.

Paso 2.5: Commit

Prompt:

```
Revisa los cambios con git status y git diff, y haz commit con un mensaje
descriptivo en español.
```

Resultado esperado del bloque 2

- `api/app/` con modelos, esquemas Pydantic y routers por entidad.
- `api/alembic/` con la migración inicial y la del rol de solo lectura.
- Script de datos de ejemplo y migraciones que se aplican al arrancar.
- `.mcp.json` con el servidor de base de datos.
- Swagger con todos los endpoints funcionando, incluida la prueba negativa.

6. Solución de problemas comunes

Ante cualquier error, el primer paso es el mismo: **copia el mensaje completo de la terminal y pégaselo a Claude tal cual**, sin resumirlo. Estos son los problemas que ya aparecieron en la prueba piloto.

> **¿Atrapado en la consola?**
>
> - Ves `:` o `(END)` → presiona `q`
> - Ves `labpresta=#` (indicador de psql) → escribe `\q` y Enter
> - Nada responde → `Ctrl + C` y después lo anterior

Síntoma	Causa	Solución
Docker Desktop no arranca o pide actualizar WSL (Windows)	WSL 2 no instalado o desactualizado, o virtualización apagada	En PowerShell como administrador: `wsl --install` o `wsl --update`, y reinicia. Verifica la virtualización en el Administrador de tareas; si está deshabilitada, actívala en el BIOS
El instalador de Node.js se queda en *"Installing visualstudio2026buildtools..."* (Windows)	Se marcó la casilla *"Automatically install the necessary tools"*	Cierra esa ventana de PowerShell y termina el proceso *Visual Studio Installer* en el Administrador de tareas. Node.js ya quedó instalado; las Build Tools no se necesitan
`port is already allocated` al levantar el stack	Otro programa usa ese puerto (por ejemplo, un PostgreSQL instalado localmente en el 5432)	Cambia el puerto del host en `.env` (por ejemplo 5434 o 8081) y vuelve a levantar
Los cambios en el código de la API no se reflejan	El reload no detecta cambios en carpetas de Windows	Confirma `WATCHFILES_FORCE_POLLING=true` y el volumen de `./api/app` en `docker-compose.yml`. Revisa con `docker compose logs api`
`/bin/sh^M: not found` o `entrypoint.sh: no such file or directory`	El script tiene finales de línea de Windows (CRLF)	Confirma el `.gitattributes` con `eol=lf`, ejecuta `git add --renormalize .` y reconstruye con `docker compose up --build`
Git avisa *"LF will be replaced by CRLF"*	Configuración de Git en Windows	Es informativo. El `.gitattributes` del bloque 1 lo resuelve
La API no arranca: no encuentra `.env` o una variable	Falta el archivo `.env` (no se versiona)	Copia `.env.example` como `.env`
El servidor MCP no aparece	Sesión abierta antes de crear `.mcp.json`, o servidor sin aprobar	Abre una sesión nueva y aprueba el servidor del proyecto

Síntoma	Causa	Solución
El servidor MCP aparece pero no conecta	Stack apagado, Docker Desktop cerrado, o dirección incorrecta	Levanta el stack, confirma que la dirección es `host.docker.internal:5433` y pide a Claude que pruebe el comando de `.mcp.json` en la terminal
Claude dice que alcanzaste el límite de uso	Sesiones largas y tareas muy grandes	Espera a que se renueve la ventana de uso. Después: una sesión por tarea, `/clear` entre tareas y prompts acotados

7. Lista de verificación y evidencia

Marca cada punto antes de dar por terminado un bloque. La evidencia se entrega en el repositorio y en la bitácora de prompts.

Tarea previa

- [ ] Captura de `git --version`, `docker --version`, `docker compose version` y `docker run hello-world`
- [ ] Captura de Claude Code respondiendo qué archivos hay en la carpeta
- [ ] Repositorio con `docs/` (plan) y `prototipo/`

Bloque 1

- [ ] `CLAUDE.md` con las seis secciones, revisado por ti
- [ ] `docker compose up --build` levanta `db`, `api` y `web` sin errores
- [ ] `localhost:8080`, `localhost:8000/docs` y `localhost:8080/api/health` responden
- [ ] El reload funciona sin reconstruir la imagen
- [ ] `.env` fuera del repositorio; `.env.example` dentro
- [ ] Al menos tres commits: insumos, `CLAUDE.md`, estructura base

Bloque 2

- [ ] Migración inicial revisada con la lista del paso 2.1
- [ ] `\dt` y `\d <tabla>` coinciden con el plan
- [ ] CRUD de cada entidad probado en Swagger
- [ ] Cada regla de negocio probada, incluida una prueba negativa con respuesta 4xx
- [ ] Endpoint simulado de IA, si aplica
- [ ] MCP aprobado; la consulta funciona y el intento de modificar falla
- [ ] Commit del bloque

Bitácora de prompts

Crea `docs/bitacora_prompts.md` y registra cada paso con esta estructura:

Paso	Prompt usado	Qué hizo Claude	Qué corregí o pedí cambiar
1.5	Estructura base en modo Plan	Propuso compose con 3 servicios	Pedí puerto 5433 porque tenía PostgreSQL local

La última columna es la más importante: demuestra que revisaste el trabajo del agente.